ЗВІТ

до лабораторної роботи № 1.3

«Використання керуючих конструкцій розгалуження та циклів при роботі з одновимірними масивами»

Варіант 26

Виконав:	Трохимчук Іван Васильович
Група:	ІО-62
Викладач:	Русінов Володимир В.

2026

1. Мета роботи

Набути практичних навичок створення алгоритмів із використанням розгалужень, циклів та одновимірних масивів; сформувати масив Y, обчислити масив Z за заданою кусочною функцією та знайти максимальне значення результуючого виразу.

2. Постановка задачі та варіант 26

Програма зчитує цілі значення n, a та b. Початковий масив формується за правилом Y[i] = -30 + 5·i для індексів i від 0 до n-1. Проміжний масив Z обчислюється за функцією № 6 з методичних вказівок:

Z[i] = 1 - a·Y[i], якщо -20 < Y[i] < 10; інакше Z[i] = 7 - b·Y[i].

Для варіанта 26 результуюча функція відповідає функції g № 2: максимум виразів виду (-1)^i·b + z_i. У наданому коді ці значення записуються в масив C, після чого програма намагається знайти максимум. Оскільки індекс у масиві починається з 0, вираз pow(-1, i)·b у першому елементі має знак +, тоді як математична формула з індексом від 1 починається зі знака −.

3. Лістинг наданої програми

Нижче наведено лістинг програми.

```
#include <stdio.h>
#include <math.h>


void printArray(const char* name, int arr[], int u) {
    printf("%s: ", name);

    for (int i = 0; i < u; i++) {
        printf("%d ", arr[i]);
    }

    printf("\n");
}

int main() {
    int a;
    int b;
    int r;
    unsigned int n;
    printf("Enter number n: ");
    scanf("%d", &n);
    printf("Enter number a: ");
    scanf("%d", &a);
    printf("Enter number b: ");
    scanf("%d", &b);
    int y[n];
    int z[n];
    int c[n];
    for (int i = 0; i < n; i++) {
        y[i] = -30 + 5 * i;
    }
    for (int i = 0; i < n; i++) {
        if (-20 < y[i] && y[i] < 10) {
            z[i] = 1 - a * y[i];
        }
        else {
            z[i] = 7 - b * y[i];
        }
    }
    for (int i = 0; i < n; i++) {
        c[i] = pow(-1, i) * b + z[i];
    }

    r = c[0];

    for (int i = 1; i < n; i++) {
        if (c[i] > r) {
            r = c[i];
        }
    }

    printArray("Y", y, n);
    printArray("Z", z, n);
    printArray("R", c, n);


    printf("MAX: %d", r);

    return 0;
}
```

4. Блок-схема програми

Для пошуку максимального значення змінній r присвоюється перший елемент масиву C, після чого послідовно перевіряються решта елементів.

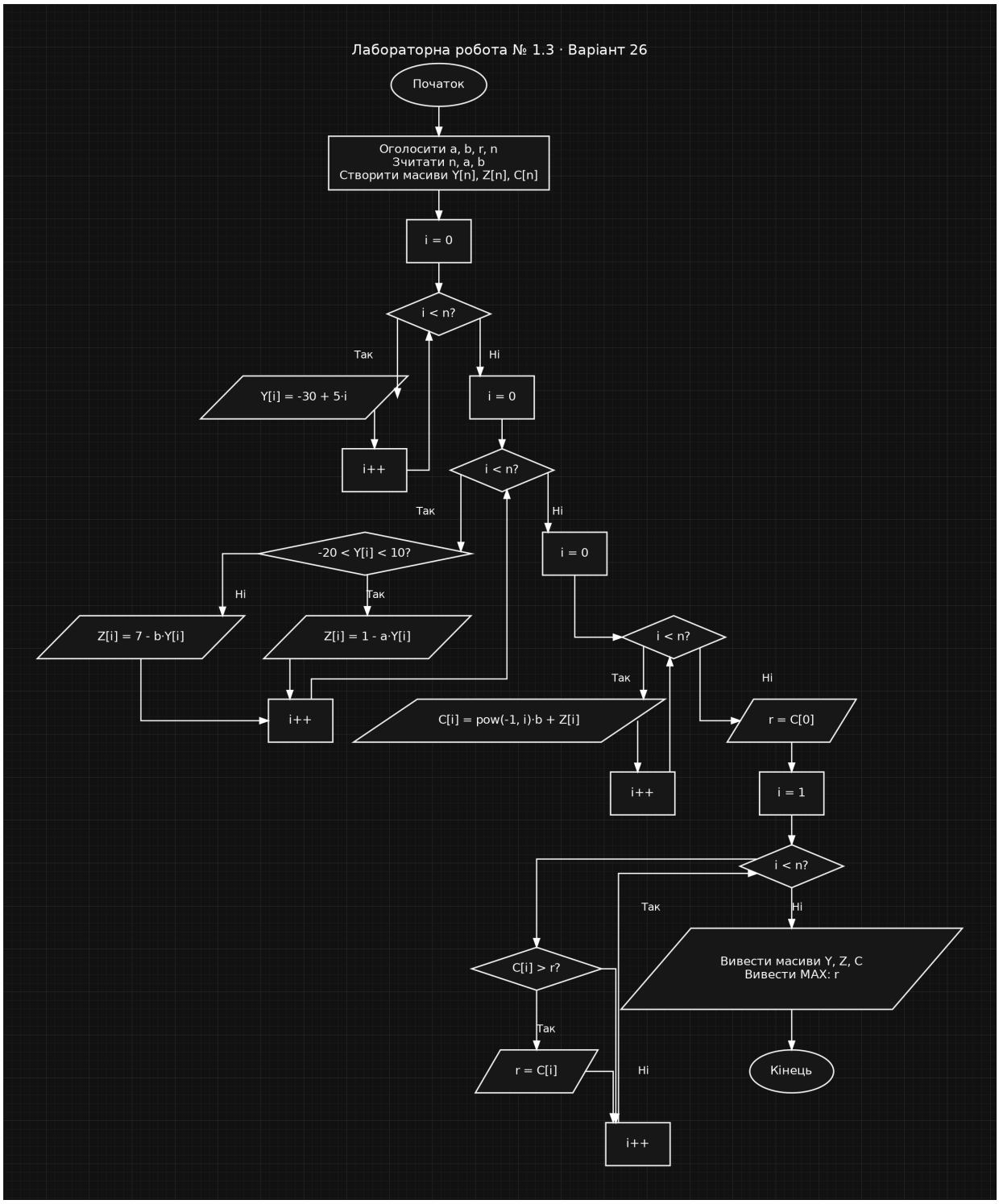


Рисунок 1 - Блок-схема алгоритму пошуку максимального значення.

5. Приклад розрахунку

Для перевірки проміжних обчислень використаємо n = 9, a = 2, b = 3. За формулою початкового масиву одержуємо Y = [-30, -25, -20, -15, -10, -5, 0, 5, 10]. Значення на межах -20 та 10 не належать відкритому інтервалу (-20, 10), тому для них використовується друга гілка функції.

i (індекс)	Y[i]	Z[i]	C[i] за кодом
0	-30	97	100
1	-25	82	79
2	-20	67	70
3	-15	31	28
4	-10	21	24
5	-5	11	8
6	0	1	4
7	5	-9	-12
8	10	-23	-20

Масиви, які будуть виведені функцією printArray для цього прикладу:

Y: -30 -25 -20 -15 -10 -5 0 5 10

Z: 97 82 67 31 21 11 1 -9 -23

R: 100 79 70 28 24 8 4 -12 -20 (у програмі підпис R відповідає масиву C)

Для цього прикладу програма виведе MAX: 100.

6. Алгоритм пошуку максимального значення

Спочатку змінній r присвоюється значення першого елемента масиву C. Далі цикл послідовно порівнює r з елементами C[1], C[2] і так далі. Якщо поточний елемент більший за r, значення r оновлюється. Після завершення циклу r містить максимальне значення масиву C. Алгоритм передбачає, що n >= 1.

7. Висновок

У роботі розглянуто формування одновимірного масиву Y, обчислення масиву Z за заданою умовою та формування масиву C. Максимальне значення визначається послідовним порівнянням елементів масиву. Для прикладу з n = 9, a = 2 і b = 3 отримано MAX = 100.